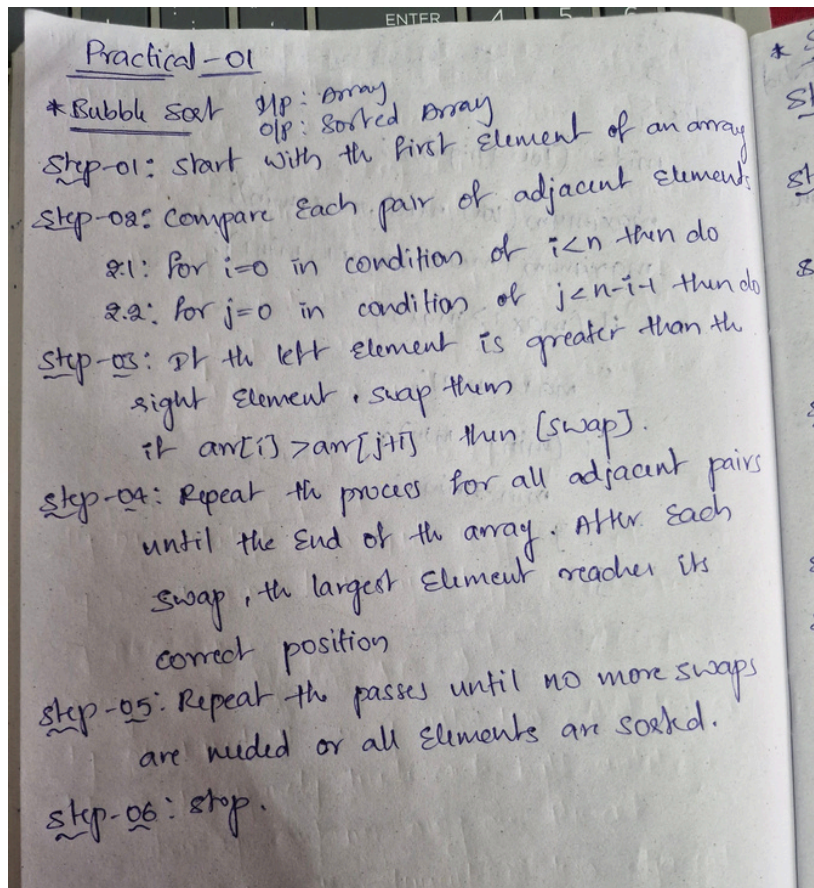


Practical: 1	Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort
29/07/2026	

Aim: To Implement and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

1.Bubblesort:

Algorithm:



Program:

```
#include <iostream>
#include <vector>

using namespace std;

void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - i - 1; ++j) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
```

```
        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
    }
}
}

int main() {
    int size;
    cout << "Enter the number of elements: ";
    cin >> size;

    vector<int> arr(size);
    cout << "Enter " << size << " integers: ";
    for (int i = 0; i < size; ++i) {
        cin >> arr[i];
    }

    bubbleSort(arr);

    cout << "Sorted array: ";
    for (int i = 0; i < size; ++i) {
        cout << arr[i] << " ";
    }
    cout << endl;

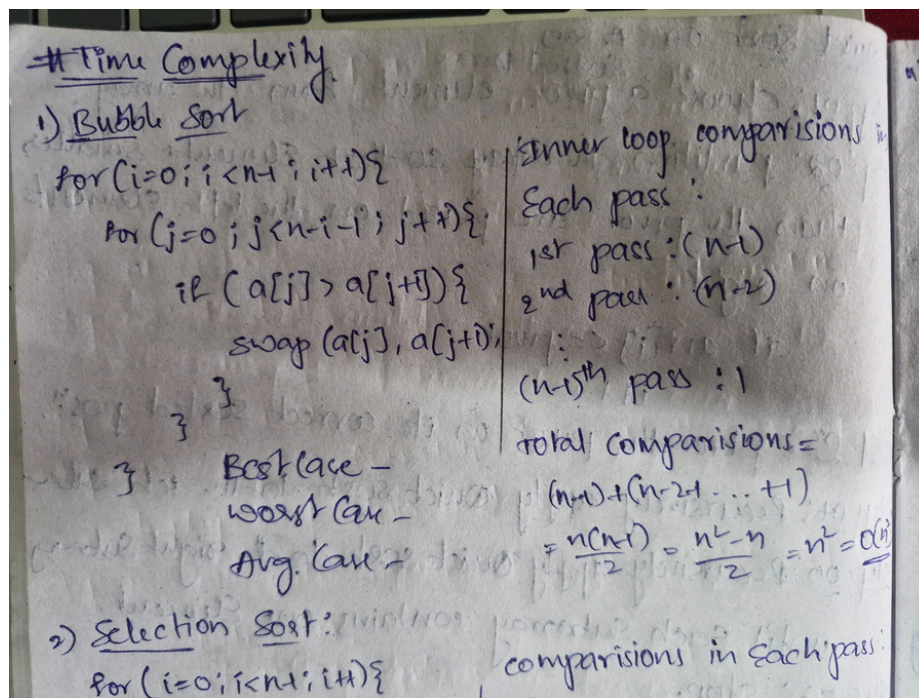
    return 0;
}
```

Output:



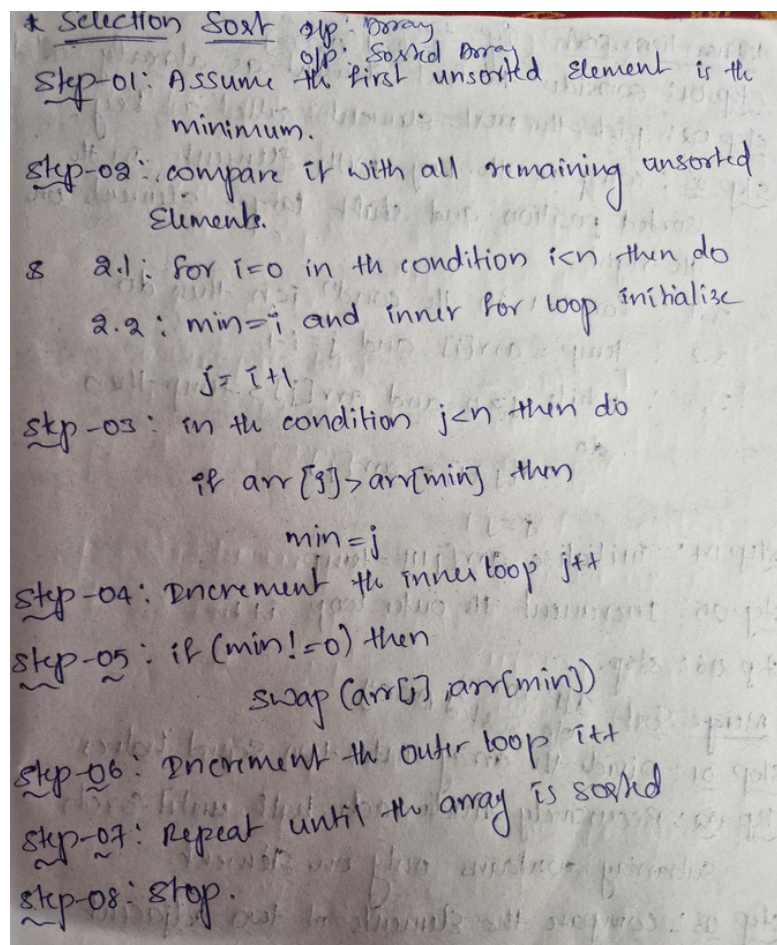
```
PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ x.cpp -o x } ; if ($?) { .\x }
Enter the number of elements: 5
Enter 5 integers: 5 1 10 3 2
Sorted array using bubblesort : 1 2 3 5 10
PS C:\Users\karth\Desktop\DSA LEARNING> |
```

Time Complexity:



2. Selection sort:

Algorithm:



Program:

```
#include <iostream>
#include <vector>

void selectionSort(std::vector<int>&arr) {
    int n = arr.size();
    for(int i=0; i<n-1; ++i){
        int minIndex = i;
        for(int j=i+1; j<n; ++j){
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        if (minIndex != i) {
            std::swap(arr[i], arr[minIndex]);
        }
    }
}

int main() {
    int n;
    std::cout << "Enter the number of elements: ";
    std::cin >> n;

    if(n <= 0) {
        return 0;
    }

    std::vector<int> arr(n);
    std::cout << "Enter " << n << " integers: ";
    for (int i = 0; i < n; ++i) {
        std::cin >> arr[i];
    }

    selectionSort(arr);

    std::cout << "Sorted array: ";
    for (int i = 0; i < n; ++i) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

Output:

```
PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ x.cpp -o x } ; if ($?) { .\x }
Enter the number of elements: 4
Enter 4 integers: 0 1 5 2
Sorted array using selection sort: 0 1 2 5
PS C:\Users\karth\Desktop\DSA LEARNING> |
```

Time complexity:

Worst Case - $\frac{n(n-1)}{2} = \frac{n^2 - n}{2} = n^2 = O(n^2)$
Avg. Case - $\frac{n(n-1)}{2} = \frac{n^2 - n}{2} = n^2 = O(n^2)$

2) Selection Sort:

```
for (i=0; i<n; i++) {
    min=i;
    for (j=i+1; j<n; j++) {
        if (a[j] < a[min])
            min=j;
    }
    swap(a[i], a[min]);
}
```

comparisons in each pass:
1st pass : $n-1$
2nd pass : $n-2$
:
($n-1$)th pass : 1
Total comparisons = $(n-1) + (n-2) + \dots + 1$
 $= \frac{n(n-1)}{2} = \frac{n^2 - n}{2} = n^2 = O(n^2)$

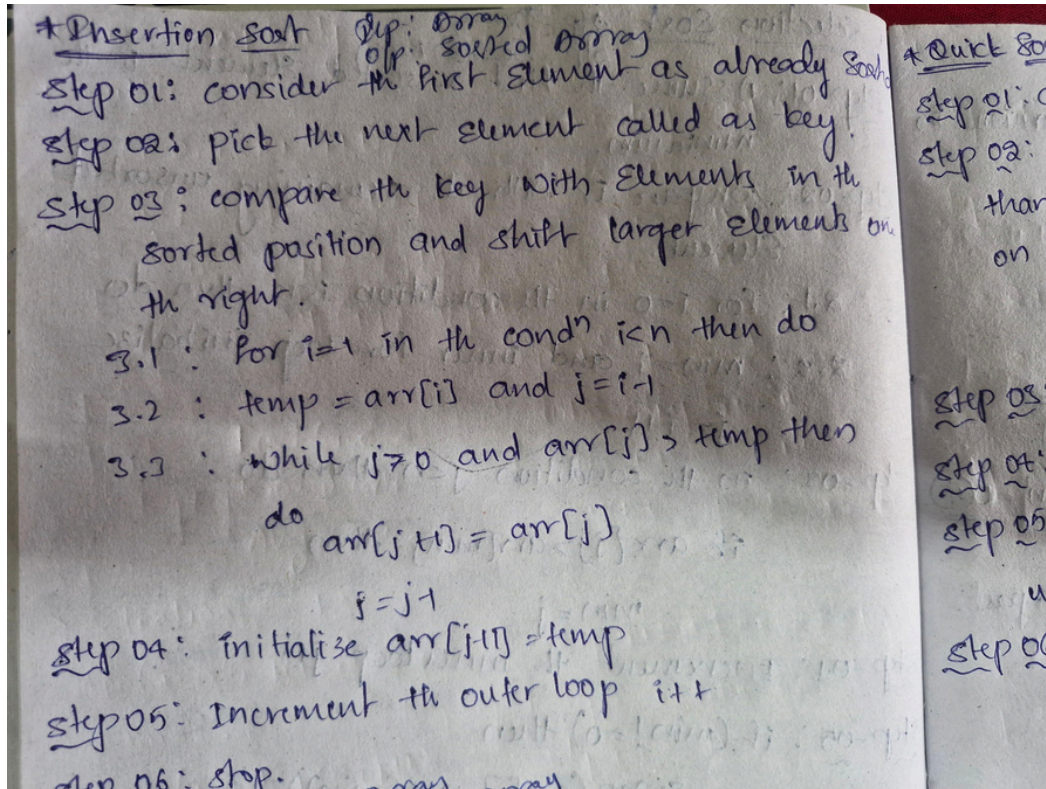
3) Insertion Sort:

```
for (i=1; i<n; i++) {
    // ...
}
```

Best Case - $O(n)$
Worst Case - $O(n^2)$
Avg Case - $O(n^2)$
comparisons = $1+2+3+\dots+n-1$

Insertion sort:

Algorithm:



Program:

```
#include <iostream>
#include <vector>
void insertionSort(std::vector<int>& arr) {
    int n = arr.size();
    for(int i=1; i<n; ++i) {
        int key = arr[i];
        int j=i-1;
        while(j>=0 && arr[j] > key) {
            arr[j+1]=arr[j];
            j=j-1;
        }
        arr[j + 1] = key;
    }
}

int main() {
    int n;
    std::cout<<"Enter the number of elements: ";
    std::cin >> n;
```



```

if(n <= 0) {
    return 0;
}

std::vector<int> arr(n);
std::cout << "Enter " << n << " integers: ";
for(int i = 0; i < n; ++i) {
    std::cin >> arr[i];
}

insertionSort(arr);

std::cout << "Sorted array: ";
for(int i = 0; i < n; ++i) {
    std::cout << arr[i] << " ";
}
std::cout << std::endl;

return 0;
}

```

Output:

```

PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ x.cpp -o x } ; if ($?) { .\x }
Enter the number of elements: 4
Enter 4 integers: 10 2 5 6
Sorted array: 2 5 6 10
PS C:\Users\karth\Desktop\DSA LEARNING>

```

Time complexity:

$\text{swap}(a[j], a[\text{min}]);$
 $\text{Total comparisons} = (n-1) + (n-2) + \dots + 1$
 $= \frac{n(n-1)}{2} = \frac{n^2 - n}{2} = n^2 - O(n)$
Best Case -
Worst Case -
Avg Case -

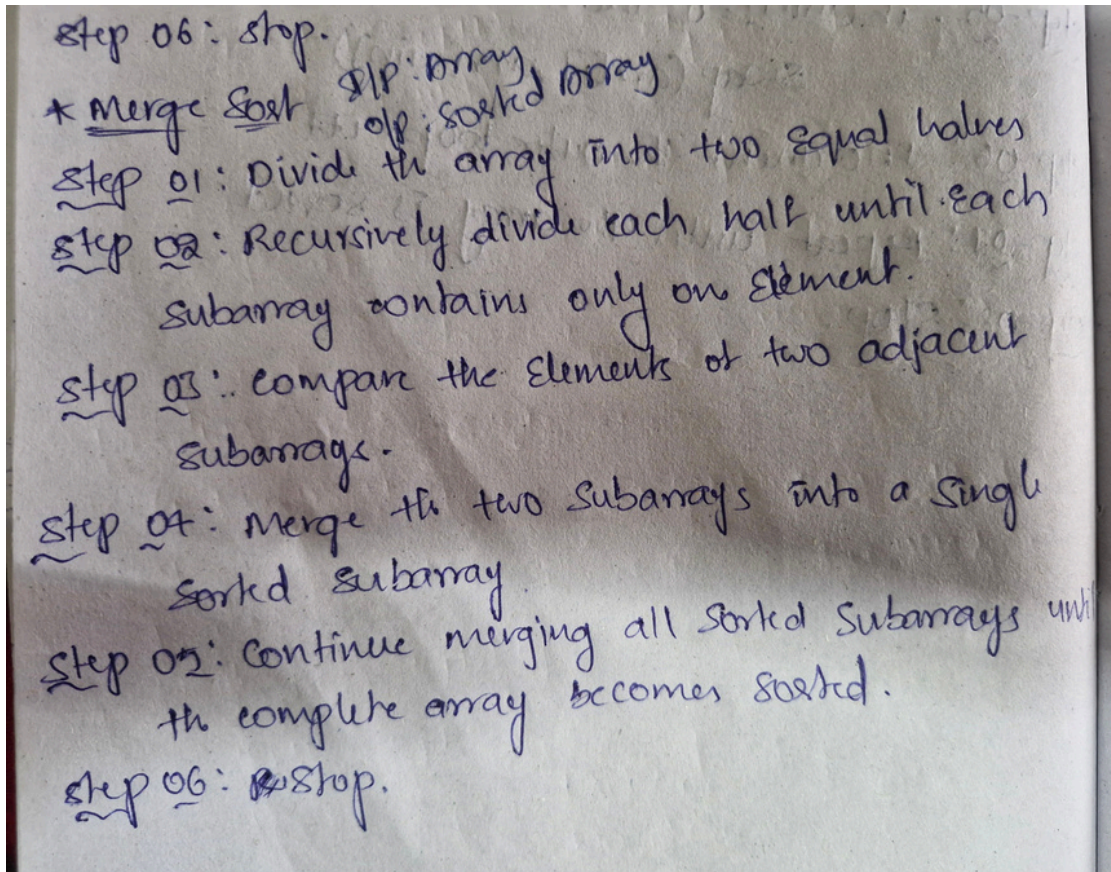
3) Insertion Sort:
 $\text{for}(i=1; i \leq n; i++) \{$
 $\text{key} = a[i];$
 $j = i-1;$
 $\text{while}(j > 0 \ \&\& \ a[j] > \text{key}) \{$
 $\text{a}[j+1] = a[j];$
 $j--;$
 $\}$
 $\text{a}[j+1] = \text{key};$
 $\}$

$\text{comparisons} = 1 + 2 + 3 + \dots + n-1$
 $= \frac{n(n-1)}{2}$
 $= \frac{n^2 - n}{2}$
 $= n^2$
Best Case -
Worst Case -
Avg Case -

$O(n^2)$

4. Merge sort:

Algorithm:



Program:

```
#include <iostream>
#include <vector>
```

```
void merge(std::vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - left;

    std::vector<int> L(n1);
    std::vector<int> R(n2);

    for (int i = 0; i < n1; ++i) {
        L[i] = arr[left + i];
    }
    for (int j = 0; j < n2; ++j) {
        R[j] = arr[mid + 1 + j];
    }

    int i = 0;
```



```
int j = 0;
int k = left;

while(i<n1&& j<n2) {
    if (L[i] <= R[j]) {
        arr[k] = L[i];
        ++i;
    } else {
        arr[k] = R[j];
        ++j;
    }
    ++k;
}

while (i < n1) {
    arr[k] = L[i];
    ++i;
    ++k;
}

while (j < n2) {
    arr[k] = R[j];
    ++j;
    ++k;
}
}

void mergeSort(std::vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    int n;
    std::cout << "Enter the number of elements: ";
    std::cin >> n;

    if (n <= 0) {
        return 0;
    }
}
```

```
std::vector<int> arr(n);
std::cout << "Enter " << n << " integers: ";
for(int i = 0; i < n; ++i) {
    std::cin >> arr[i];
}

mergeSort(arr, 0, n - 1); std::cout

<< "Sorted array: ";
for(int i = 0; i < n; ++i) {
    std::cout << arr[i] << " ";
}
std::cout << std::endl;

return 0;
}
```

Output:

```
PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ x.cpp -o x } ; if ($?) { .\x }
Enter the number of elements: 4
Enter 4 integers: 10 2 5 1
Sorted array: 1 2 5 10
PS C:\Users\karth\Desktop\DSA LEARNING>
```

Time complexity:

1) Merge sort:

```

Merge sort(int arr[], left, right) {
    if (left < right) {
        mid = (left + right) / 2;
        merge sort [arr, left, mid];
        merge sort [arr, mid + 1, right];
        merge [arr, left, mid, right]; // - log 2n
    }
}

```

$T(n) = O(n) \times O(\log_2 n)$
 $= O(n \log n)$

Best case -
 worst case -
 Avg. case -

2) Quick sort:

```

Quick sort (arr[], low, high)

```

5. Quick sort:

Algorithm:

* Quick sort on array
step 01: choose a pivot element from the array.
step 02: partition the array so that elements smaller than the pivot are placed on the left elements on right.
 $if\ arr[i] < pivot$
 $it++$
step 03: place the pivot in its correct sorted posⁿ.
step 04: Recursively apply quick sort to the left subarray.
step 05: Recursively apply quick sort to the right subarray.
 until each subarray contains one element.
step 06: stop.

Program:

```
#include <iostream>
#include <vector>

void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

int partition(std::vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
```

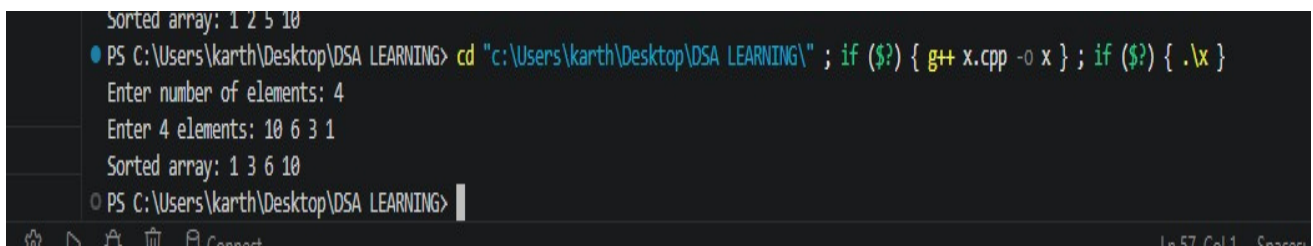


```
for(int j=low; j< high; j++) {
    if(arr[j]<pivot) {
        i++;
        swap(arr[i],arr[j]);
    }
}
swap(arr[i+1],arr[high]);
return i + 1;
}

void quickSort(std::vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi=partition(arr, low, high);
        quickSort(arr,low, pi - 1);
        quickSort(arr,pi + 1, high);
    }
}

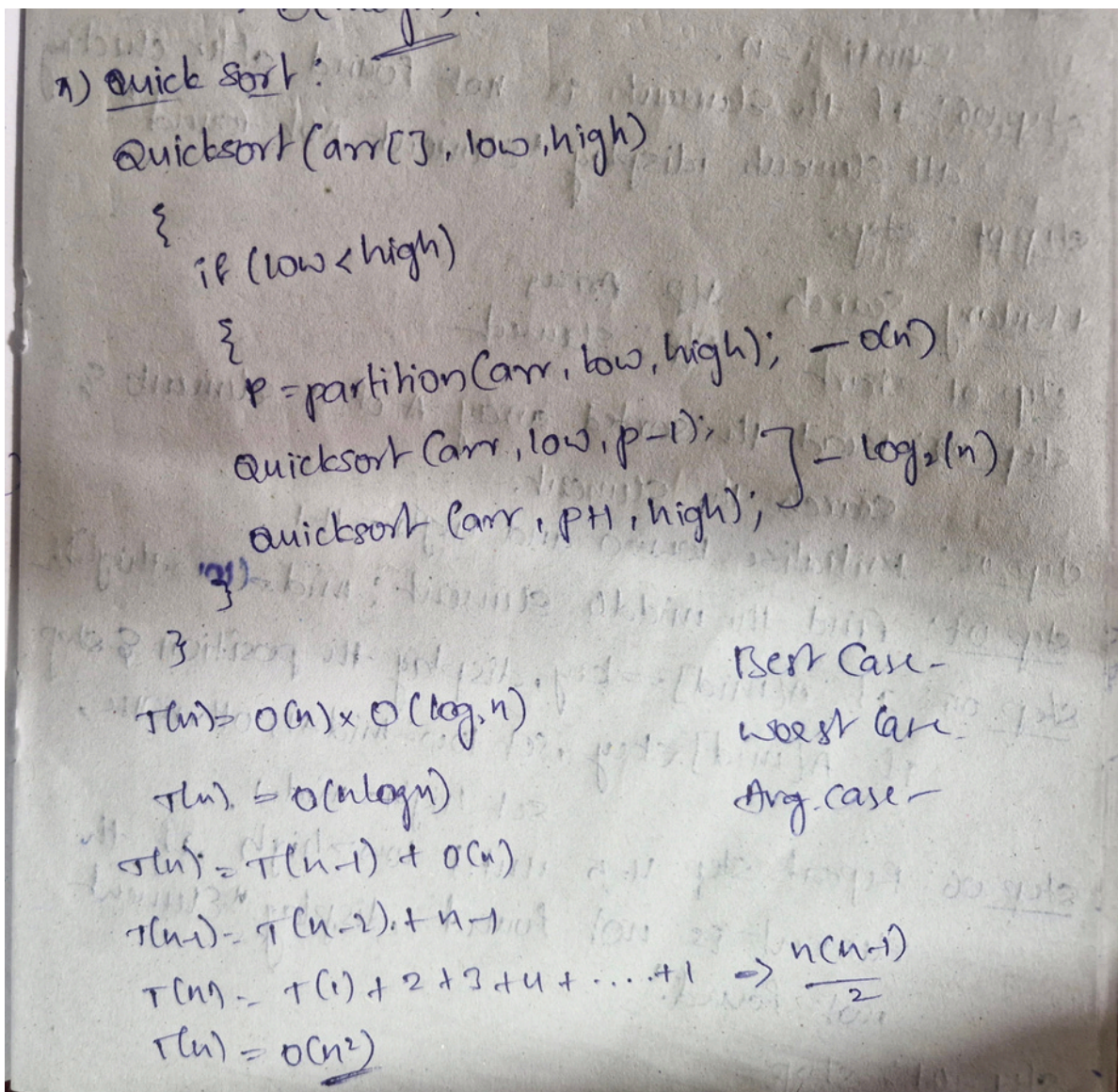
int main() {
    int n;
    std::cout<<"Enter number of elements: ";
    std::cin >> n;

    std::vector<int> arr(n);
    std::cout<<"Enter " << n << " elements: ";
    for(int i=0; i<n; i++) {
        std::cin>>arr[i];
    }
    quickSort(arr,0,n - 1);
    std::cout<<"Sorted array: ";
    for(int i=0; i<n; i++) {
        std::cout<<arr[i] << " ";
    }
    std::cout<<std::endl;
    return 0;
}
```

Output:

```
Sorted array: 1 2 5 10
PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ x.cpp -o x } ; if ($?) { .\x }
Enter number of elements: 4
Enter 4 elements: 10 6 3 1
Sorted array: 1 3 6 10
PS C:\Users\karth\Desktop\DSA LEARNING>
```

Time complexity:



conclusion :

Hence, By Implementing and analysing of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quick sort.

The time complexity plays an important role in the logical part of the program. And the Programs are successfully executed.